

ТИТУЛЬНЫЙ ЛИСТ

По дисциплине пм 01 мдк 01 мдк 02

Разветртыване LLM сервера для разработки

СОДЕРЖАНИЕ

Титульный лист	1
Содержание	2
Термины и определения	3
Перечень сокращений и обозначений	4
Введение	5
1 Анализ решений для локального развёртывания LLM и обоснование выбора llama.cpp	6
1.1 Ollama	7
1.2 Llama.cpp	8
1.3 Вывод анализа	9
2 Определение минимальных аппаратных требований к серверной платформе	9
2.1 Факторы, определяющие аппаратные требования	9
2.2 Расчёт требований к оперативной памяти	10
2.3 Требования к процессору	10
2.4 Требования к дисковому пространству	11
2.5 Сетевые требования	11
2.6 Итоговые минимальные требования для курсовой работы	11
3 РАЗРАБОТКА КОНФИГУРАЦИИ СЕРВЕРА	12
3.1 Подготовка программного окружения	12
3.2 Скрипт запуска сервера	13
3.3 Работа модели	14
4 РЕАЛИЗАЦИЯ ИНТЕГРАЦИИ ЛОКАЛЬНОГО LLM-СЕРВЕРА СО СРЕДОЙ РАЗРАБОТКИ CODE OSS	14
4.1 Выбор инструмента интеграции: расширение Continue	14
4.1 Настройка подключения к локальному серверу	15
4.2 Проверка работоспособности интеграции	16

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей курсовой работе (проекте) применяют следующие термины с соответствующими определениями:

API (Application Programming Interface) - программный интерфейс, то есть описание способов взаимодействия одной компьютерной программы с другими.

AI - также ИИ, искусственный разум, в самом широком смысле — научно-технологическая область, занимающаяся изучением и созданием интеллектуальных сущностей.

LLM (large language model) - Большая языковая модель основанная на нейронной сети с множеством параметров (миллиарды весовых коэффициентов и более), которая проходит предварительное обучение на обширных массивах неразмеченного текста методами самообучения.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей курсовой работе (проекте) применяют следующие сокращения и обозначения:

ВВЕДЕНИЕ

Современные IT-компании активно внедряют инструменты искусственного интеллекта в процессы разработки. Согласно исследованию Stack Overflow 2025 года, 84% разработчиков используют или планируют использовать AI-инструменты, при этом 51% применяют их ежедневно. Исследование Complexity Science Hub показывает, что доля AI-ассистированного кода на GitHub выросла с 5% в 2022 году до 29% к концу 2024 года.

Однако рост популярности облачных AI-решений (GitHub Copilot, ChatGPT) обостряет проблему безопасности интеллектуальной собственности. Передача фрагментов кодовой базы сторонним серверам создаёт риски утечки конфиденциальных алгоритмов и коммерческой тайны. Эксперты отмечают, что при использовании облачных AI-инструментов возникают: размывание границ суверенитета данных, неконтролируемость цепочек вывода и расширение поверхности атаки.

Актуальность работы определяется тремя факторами. Во-первых, необходимостью защиты кодовой базы: облачные AI-сервисы могут использовать переданный код для дообучения моделей, что создаёт риски раскрытия интеллектуальной собственности. Во-вторых, потребностью в автономности разработки при ограниченном доступе в Интернет или работе в закрытых периметрах. В-третьих, экономической целесообразностью: локальное развёртывание позволяет избежать регулярных абонентских платежей.

Существует ряд open-source инструментов для локального запуска моделей (Ollama, llama.cpp, vLLM). Однако проблема системного подхода к интеграции такого сервера в реальную рабочую среду организации, с учётом требований администрирования и удобства для конечных пользователей, остаётся недостаточно проработанной.

Объект и предмет исследования:

- 1) Объект исследования: процесс организации безопасной локальной инфраструктуры для использования LLM в разработке ПО.
- 2) Предмет исследования: методы развёртывания, администрирования и интеграции локального LLM-сервера (на базе llama-server) в рабочую среду IT-специалистов, в частности - подключение к Code OSS.

Цель работы: разработать и экспериментально подтвердить работоспособность локального LLM-сервера на базе llama-server, обеспечить его администрирование в условиях локальной сети и выполнить интеграцию со средой разработки Code OSS.

Задачи:

- 1) Провести анализ существующих решений для локального развёртывания LLM и обосновать выбор llama-server.
- 2) Определить минимальные аппаратные требования и выполнить установку и настройку llama-server в локальной сети.
- 3) Разработать конфигурацию сервера, обеспечивающую стабильную работу.
- 4) Реализовать интеграцию локального LLM-сервера с Code OSS на клиентском компьютере.
- 5) Выполнить экспериментальную проверку работоспособности решения .
- 6) Сформулировать рекомендации по администрированию сервера .

1 АНАЛИЗ РЕШЕНИЙ ДЛЯ ЛОКАЛЬНОГО РАЗВЁРТЫВАНИЯ LLM И ОБОСНОВАНИЕ ВЫБОРА LLAMA.CPP

Для организации локального LLM-сервера, доступного нескольким пользователям в корпоративной сети, наиболее актуальными являются два решения: Ollama и llama.cpp (с компонентом llama-server). Оба инструмента базируются на единой технологической основе (формат GGUF, оптимизации CPU/GPU), но принципиально различаются по архитектуре и подходу к администрированию. // вставить источники на сравнение ollama vs llama.cpp

1.1 Ollama

Ollama позиционируется как «LLM-движок для разработчиков», предлагая максимально простой путь запуска моделей. Однако при использовании в многопользовательском сценарии проявляются его некоторые недостатки

Достоинства:

- 1) Установка одной командой, готовые пакеты для всех ОС
- 2) Встроенный реестр моделей
- 3) Open-AI совместимый API

Недостатки:

- 1) Последовательная обработка запросов — при одновременных обращениях нескольких пользователей запросы ставят в очередь
- 2) Ограниченный контроль над ресурсами — администратор не может распределить потоки CPU между задачами
- 3) Отсутствие встроенного веб-интерфейса
- 4) Логирование и мониторинг — стандартные средства логирования недостаточны для реальной эксплуатации

1.2 Llama.cpp

Llama-server — это встроенный HTTP-сервер, входящий в состав проекта Llama.cpp, предназначенный для production-развёртывания языковых моделей в корпоративной среде. В отличие от Ollama, который ориентирован на индивидуальных разработчиков и простоту использования, Llama-server предоставляет администраторам организации полный спектр инструментов для управления многопользовательской инфраструктурой.

Достоинства:

- 1) Параллельная обработка сессий — поддержка конкурентных сессий с возможностью ограничения количества одновременных сессий
- 2) Полный контроль над ресурсами — управление количеством потоков CPU приоритетами, размером контекстного окна для каждого пользователя.
- 3) Встроенный веб-интерфейс позволяет наблюдать за работой сервера, просматривать историю запросов без дополнительных надстроек
- 4) Детальное логирование — все запросы и ответы логируются с возможностью настройки уровня детализации
- 5) Anthropic-совместимый API — расширяет возможности интеграции с различными клиентами.

1.3 Вывод анализа

Ollama представляет собой удобное решение для индивидуального использования или малых групп разработчиков, где допустима последовательная обработка запросов. Однако в сценарии корпоративного локального LLM-сервера, обслуживающего нескольких пользователей одновременно, его архитектурные ограничения становятся критическими.

Llama-server обеспечивает необходимый уровень контроля, масштабируемости и наблюдаемости, что делает его оптимальным выбором в качестве базового компонента для развёртывания локального LLM-сервера в организации. Более высокая сложность первичной настройки компенсируется гибкостью и предсказуемостью работы в многопользовательской среде.

2 ОПРЕДЕЛЕНИЕ МИНИМАЛЬНЫХ АППАРАТНЫХ ТРЕБОВАНИЙ К СЕРВЕРНОЙ ПЛАТФОРМЕ

Для успешного развёртывания локального LLM-сервера на базе Llama.cpp необходимо определить минимальные аппаратные требования, обеспечивающие стабильную работу в многопользовательской среде. В отличие от индивидуального использования, где допустимы периодические задержки, серверная инфраструктура организации должна гарантировать предсказуемое время отклика при одновременной работе нескольких пользователей.

2.1 Факторы, определяющие аппаратные требования

Аппаратные требования к LLM-серверу определяются тремя основными группами факторов:

- 1) Характеристика модели — основной объём потребляемой памяти зависит от выбранной модели и типа квантования. Однако при загрузке в оперативную память требуется дополнительное пространство для KV-кэша, размер которого пропорционален контекстному окну.
- 2) Многопользовательская нагрузка. При параллельной обработке запросов от нескольких пользователей требования к памяти, GPU и CPU возрастают пропорционально количеству одновременных сессий. Каждый параллельный слот требует выделения собственного KV-кэша
- 3) Размер контекстного окна. Увеличение контекстного окна существенно повышает потребление памяти.

2.2 Расчёт требований к оперативной памяти

Общий объём оперативной памяти для llama-server можно рассчитать по формуле 1:

$$\text{RAM_total} = \text{Model_size_loaded} + \text{KV_cache_size} \times \text{N_parallel_slots} + \text{Overhead}$$

Согласно документации llama.cpp, для базового запуска достаточно 4 ГБ ОЗУ

2.3 Требования к процессору

llama.cpp эффективно работает как на x86_64, так и на AArch64 архитектурах.

Для многопользовательских сценариев на 32-ядерных серверах рекомендуется использовать CPU affinity и NUMA-aware оптимизации. При использовании GPU достаточно 2-4 ядер CPU для управления передачей данных.

2.4 Требования к дисковому пространству

Дисковое пространство необходимо для хранения файлов моделей и логов сервера:

- 1) Модель, вес зависит от кол-ва параметров
- 2) Логирование: лучше выделить 5-10 ГБ для хронологических логов
- 3) Системные требования ОС: 20 - 40 ГБ

Рекомендуемый объем: от 32ГБ + вес модели.

2.5 Сетевые требования

Для работы в локальной сети организации серьезных требований нет. Пропускная способность от 100 Мбит/с — этого достаточно для передачи текстовых данных. Для упрощения настройки рекомендуется статический IP адрес. Сервер привязывается к порту (по умолчанию 8080) и интерфейсу 0.0.0.0 для обеспечения доступности из сети.

2.6 Итоговые минимальные требования для курсовой работы

Для личного эксперимента я принял решение развернуть локальный LLM-сервер на старом, вышедшем из активной эксплуатации ноутбуке. Это необходимо для получения реального опыта администрирования физического сервера и интеграции его в рабочую среду. Также Запуск LLM-модели на заведомо слабом устройстве доказывает, что современные языковые модели не требуют дорогостоящих серверов или облачных GPU. Технология становится доступной даже для малых организаций с ограниченным бюджетом.

Для успешной работы на старом ноутбуке была выбрана модель Qwen3-0.6B в квантованном формате Q4_0 (размер файла ~469 МБ). Данная модель имеет всего 0,6 млрд параметров, что минимизирует потребление ОЗУ.

Важно понимать, что данная аппаратная платформа не рассчитана на работу в реальной рабочей среде. Но для целей курсовой работы — демонстрации принципиальной возможности создания локального LLM-сервера с нуля, его администрирования и интеграции в рабочую среду организации — данная платформа показательна.

3 РАЗРАБОТКА КОНФИГУРАЦИИ СЕРВЕРА

В качестве серверной платформы использовался легковесный дистрибутив antiX Linux, основанный на Debian, но принципиально не использующий systemd. Это накладывало определённые ограничения на стандартные способы управления сервисами, но одновременно демонстрировало гибкость подхода: для работы Llama-server не требуется какая-либо специфическая система инициализации — достаточно shell-скрипта и возможности запуска фонового процесса.

3.1 Подготовка программного окружения

На старый ноутбук была установлена antiX — дистрибутив, известный своей эффективной работой на устаревшем оборудовании. Выбор antiX обусловлен минимальным потреблением ресурсов (оперативная память после загрузки составляет менее 200 МБ), что оставляет максимум свободной памяти для загрузки LLM-модели.

Llama.cpp собирается из исходного кода, из репозитория проекта на GitHub. Компиляция производилась прямо на ноутбуке при помощи CMake, что делает исполняемые файлы оптимизированными для конкретного процессора.

Для работы на ограниченном оборудовании была выбрана квантованная модель Qwen3-0.6B в формате GGUF с квантованием Q4_0. Выбор модели определялся её малым размером (~469 МБ) и при этом достаточным

качеством ответов на русском языке. Модель была размещена в отдельной директории для удобства работы.

3.2 Скрипт запуска сервера

Поскольку antiX не использует systemd, стандартные механизмы создания systemd-сервисов (файлы .service) неприменимы. Вместо этого был разработан bash-скрипт, реализующий запуск llama-server в фоновом режиме.

Bash скрипт:

```
#!/bin/bash
```

```
# Путь к модели и параметры
```

```
MODEL_PATH="/home/chitak/models/qwen3-0.6b-q4_0.gguf"
```

```
HOST="0.0.0.0"
```

```
PORT="3000"
```

```
# Запуск сервера, лог
```

```
nohup /home/chitak/llama.cpp/build/bin/llama-server -m $MODEL_PATH --  
host $HOST --port $PORT -c 8192 -np 4 --parallel 2 >> /var/log/llama.log  
2>&1 &
```

```
echo "Сервер запущен с PID: $!"
```

Параметр «-c 8192» был выбран как компромиссное значение: для многих задач (автодополнение кода, ответы на вопросы по небольшому фрагменту кода) контекста в 8192 токена достаточно. Параметр «--parallel 2» позволяет серверу обрабатывать до двух запросов одновременно, что критически важно при интеграции с несколькими клиентами в локальной сети.

3.3 Работа модели

Для проверки работоспособности модели я с другого компьютера в локальной сети, открыл в браузере веб интерфейс llama-server по адресу «http://192.168.1.117:3000». В чате с моделью протестировал максимально простой тестовый промт «hello», показал на рисунке 1

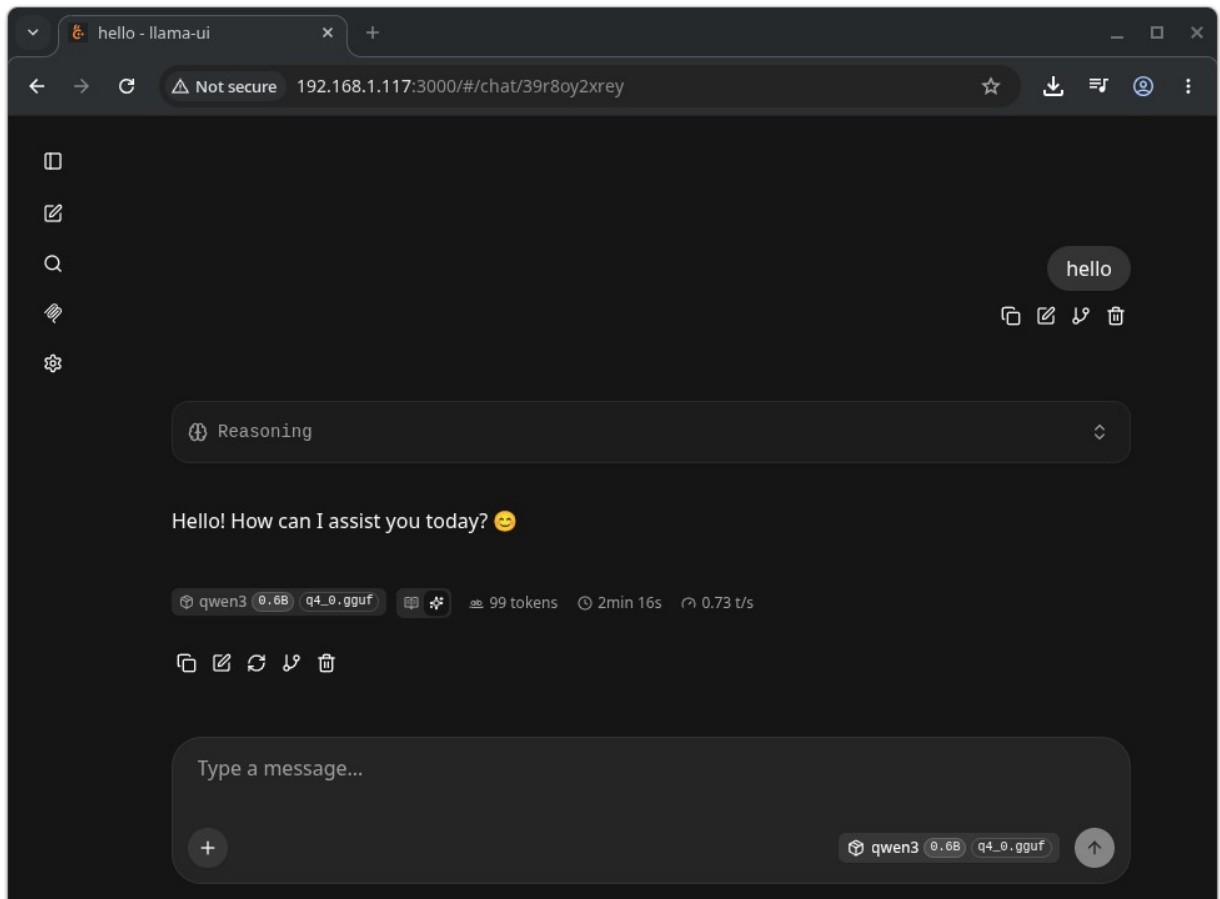


Рисунок 1: Модель отвечает на запрос

Модель успешно дала ответ на запрос, что свидетельствует о правильной работе llama-server.

4 РЕАЛИЗАЦИЯ ИНТЕГРАЦИИ ЛОКАЛЬНОГО LLM-СЕРВЕРА СО СРЕДОЙ РАЗРАБОТКИ CODE OSS

4.1 Выбор инструмента интеграции: расширение Continue

Для интеграции разработанного локального LLM-сервера в среду разработки был выбран инструмент Continue — открытое расширение (плагин) для Visual Studio Code и его форков, включая Code OSS. Continue

позиционируется как AI-ассистент для разработчиков, поддерживающий подключение к различным моделям, включая локальные

Важно отметить, что Continue доступен не только в официальном Visual Studio Code от Microsoft, но и в Code OSS — полностью открытой версии редактора, распространяемой под лицензией MIT. Это позволяет использовать расширение в среде разработки, где приоритетом является полная открытость и отсутствие телеметрии Microsoft.

4.2 Настройка подключения к локальному серверу

Continue использует конфигурационный файл в формате YAML, который по умолчанию находится в директории `~/.continue/config.yaml` (в Linux-системах). Для подключения к локальному серверу на базе llama.cpp была создана следующая конфигурация:

```
name: Local Config

version: 1.0.0

schema: v1

models:
  - name: Local llama-server
    provider: llama.cpp
    model: qwen3-0.6b-q4_0.gguf
    apiBase: http://192.168.1.117:3000
    contextLength: 8192
    roles:
      - chat
      - edit
```

- apply

tabAutocompleteModel:

provider: llama.cpp

model: qwen3-0.6b-q4_0.gguf

apiBase: http://192.168.1.117:3000

contextLength: 4096

allowAnonymousTelemetry: false

4.3 Проверка работоспособности интеграции

После сохранения конфигурации и перезагрузки Code OSS расширение Continue установило соединение с Llama-server. Успешность подключения подтверждалась появлением имени модели «Local llama-server» в интерфейсе Continue (нижняя панель чата).

Для проверки качества работы модели был отправлен простой запрос: «write a hello world program». Результат на рисунке 2.

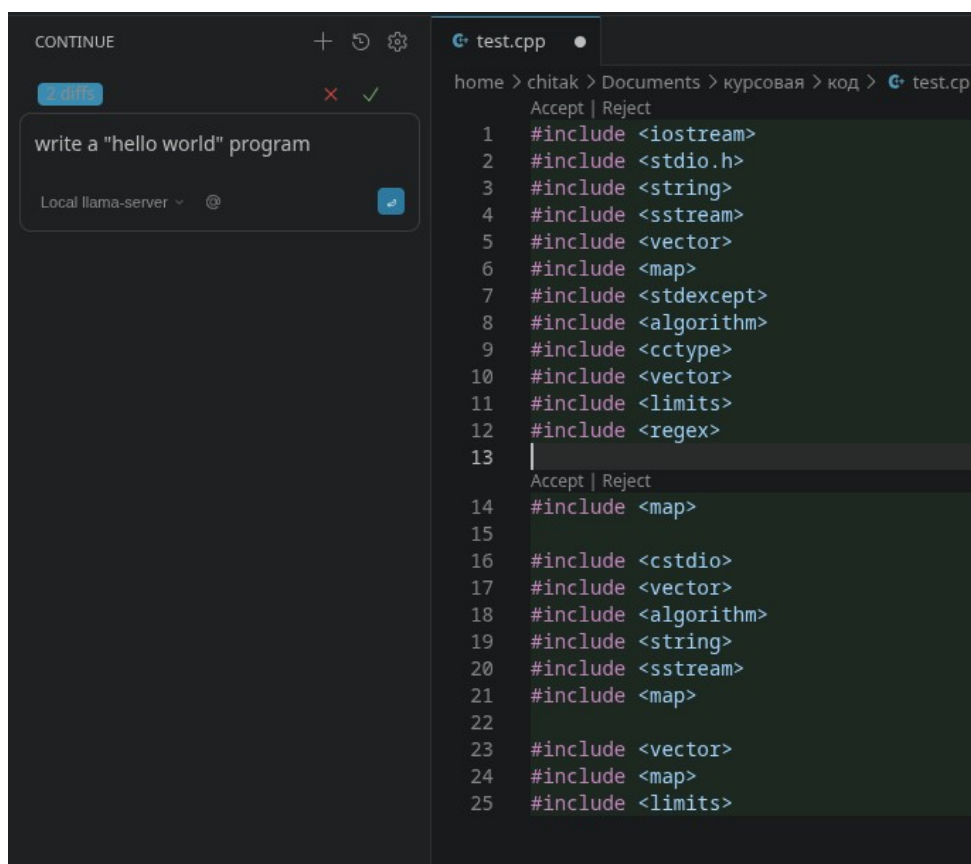
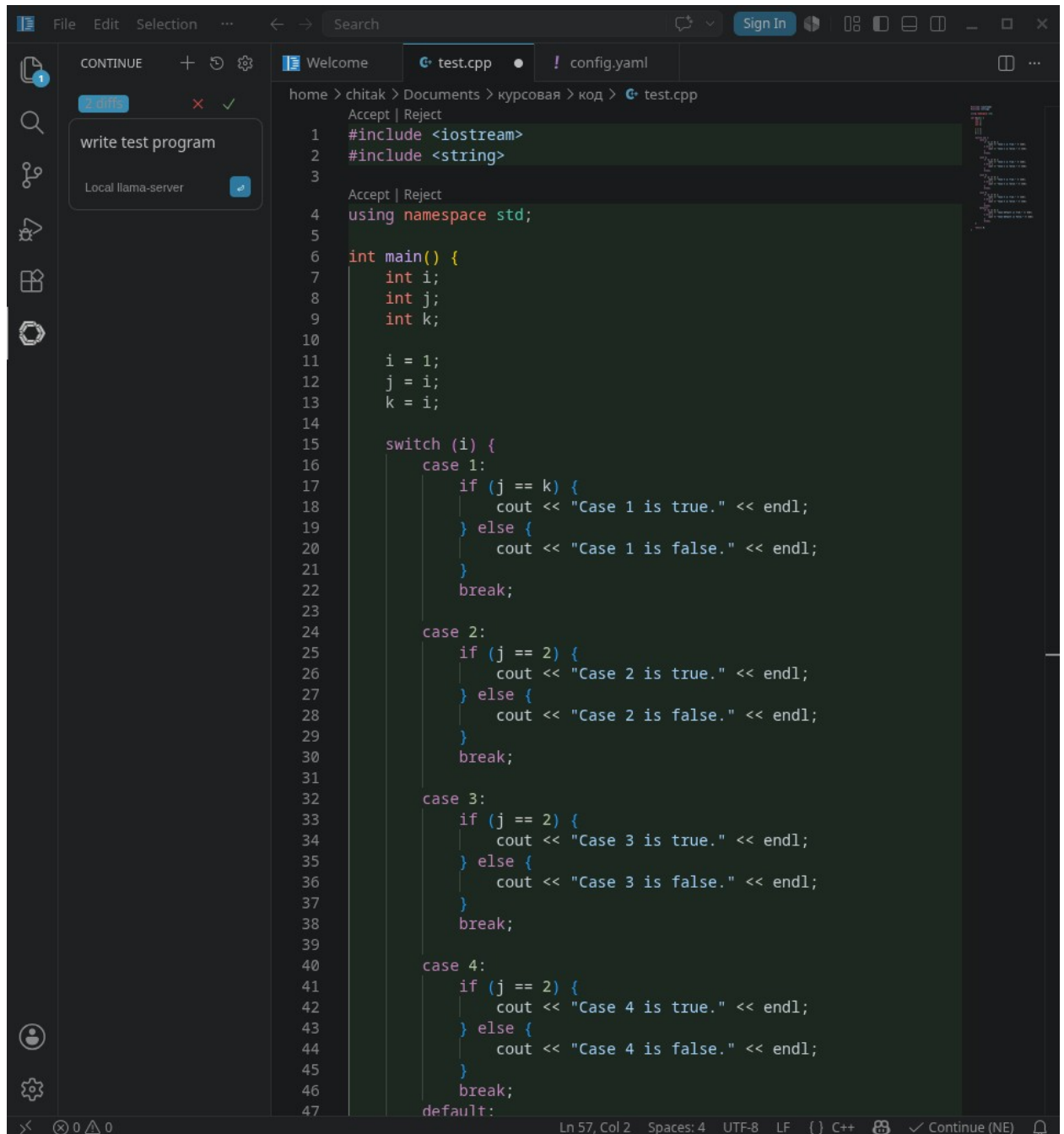


Рисунок 2: Галлюцинации модели

В данном тесте произошла галлюцинация, модель начинает генерировать множественные повторяющиеся директивы `#include`

Следующий тест с другим запросом: «write test program» оказался более успешным. В результате был получен код. Рисунок 3.



```
1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 int main() {
7     int i;
8     int j;
9     int k;
10
11     i = 1;
12     j = i;
13     k = i;
14
15     switch (i) {
16     case 1:
17         if (j == k) {
18             cout << "Case 1 is true." << endl;
19         } else {
20             cout << "Case 1 is false." << endl;
21         }
22         break;
23
24     case 2:
25         if (j == 2) {
26             cout << "Case 2 is true." << endl;
27         } else {
28             cout << "Case 2 is false." << endl;
29         }
30         break;
31
32     case 3:
33         if (j == 2) {
34             cout << "Case 3 is true." << endl;
35         } else {
36             cout << "Case 3 is false." << endl;
37         }
38         break;
39
40     case 4:
41         if (j == 2) {
42             cout << "Case 4 is true." << endl;
43         } else {
44             cout << "Case 4 is false." << endl;
45         }
46         break;
47     default:
```

Рисунок 3: Сгенерированный код

Код успешно компилируется и результат работы программы является вывод в консоль: «Case 1 is true.»

4.4 Выводы по интеграции

Интеграция локального LLM-сервера с Code OSS через расширение Continue была успешно реализована на уровне подключения: клиент (Code OSS) установил соединение с сервером (llama-server) в локальной сети и смог отправлять запросы на генерацию. Однако практическая полезность решения оказалась ограниченной из-за качества работы выбранной модели Qwen3-0.6B-Q4_0, которая на простых задачах демонстрировала галлюцинации и избыточную генерацию.

Данный результат, тем не менее, имеет ценность: он демонстрирует принципиальную достижимость цели — локальный LLM-сервер может быть интегрирован в среду разработки, заменяя облачные решения. Для повышения качества генерации потребуется использование более крупных моделей (например, Qwen2.5-Coder-7B или DeepSeek-Coder-6.7B), что, в свою очередь, потребует более мощного серверного оборудования.